Can a VLM Learn to Imagine FrozenLake?

Research in LLMs, HSE × Central University

Ivan A. Novosad
inovosad@hse.ru

Abstract

probably yes.*

1. The Bet

The assignment asks a clean question: can a VLM "imagine" the next transition, and can code use that transition for planning?

There are two different jobs here. A reactive VLM policy sees an image and directly chooses an action. A world-model VLM sees an image and an action, then predicts the next position and outcome. In Part 3, the VLM is not asked to plan. The code planner asks it four one-step questions, one per action, then chooses an action from the predicted outcomes.

My initial bet was simple. Zero-shot action-taking would probably be brittle. SFT should teach the output contract and the deterministic transition rule. GT state text should help if perception is the hard part, because then the model does not have to infer player, hole, and goal positions from pixels.

2. Setup and Output Contracts

The environment is Gymnasium FrozenLake, 8×8, deterministic, with `is_slippery=False` and RGB rendering. Maps are random but checked for reachability from start to goal. Coordinates are zero-indexed `(row, col)` from the top-left. Actions follow Gymnasium's convention: Left = 0, Down = 1, Right = 2, Up = 3.

The reactive policy had to answer with exactly one action tag:

```
<answer>Down</answer>
```

The world model had to answer with exactly one transition tag:

```
<prediction>
Position: (r, c). Outcome: safe
</prediction>
```

Valid outcomes were `safe`, `hole`, `goal`, and `wall`. The reactive system prompt was:

```
You are controlling the player in an 8x8
FrozenLake game from a rendered image.
Choose exactly one action from:
Left, Down, Right, Up.
Return only the required XML-like answer tag
and no other text.
```

The SFT world-model system prompt was:

```
You are a FrozenLake transition model.
Predict the next player position and outcome
from the rendered image and action.
Return only the required prediction tag.
```

The zero-shot world-model prompt used the same contract, with one action name inserted into a user template and the required prediction tag shown explicitly.

The reactive parser strips whitespace and applies this full-match:

```
re.fullmatch(
  r"<answer>(Left|Down|Right|Up)</answer>",
  text,
)
```

The world-model parser uses the same strict idea:

```
^<prediction>
Position: \((\d+), (\d+)\)\. Outcome:
(safe|hole|goal|wall)
</prediction>$
```

I used strict parsers. That was not decoration. In a tool-using setup, code consumes the model output. If the model returns prose or a near-miss string, the caller has to decide whether to trust it. Here I followed the assignment contract: malformed output is non-compliant. The deterministic fallback action is Right, action ID 2.

* source of well-known anecdote: https://arxiv.org/abs/1110.2832

Table 1. Evaluation seed sets. The held-out planning set is the fair five-condition comparison.

Use	Seeds	Why
Preliminary reactive run	0–29	Original Part 1 run
SFT train maps	0–79	SFT train split
SFT validation maps	80–99	Held out from train
Planning comparison	100–129	Held out and shared

Table 2. Part 1 reactive baseline on preliminary seeds 0–29.

Condition	Succ.	Hole	Trunc.	Steps	Format
Reactive VLM	0.000	0.900	0.100	13.10	0.000

This became one of the main results. Zero-shot Qwen often gave semantically close strings, but not the required tags.

3. Part 1: Reactive Baseline

3.1. Hypothesis

The reactive baseline was the cold-start test. I expected it to be weak because the model only sees the current rendered image. I also expected the answer format to fail before the policy had much chance to matter.

3.2. Design

The model receives the current image and chooses one action from Left, Down, Right, Up. No ground-truth state, no hole list, no coordinates, no hidden map text. The parser accepts only exact tags like `<answer>Down</answer>`.

I also implemented a ground-truth greedy planner. It knows the exact player, goal, and hole positions. At each state it rejects actions predicted as `hole` or `wall`, then chooses the remaining action that minimizes Manhattan distance to the goal. This is not a complete planner. It is the same one-step greedy idea used as the GT one-step-greedy ceiling in Part 3.

3.3. Results

The real Qwen reactive baseline ran on 30 preliminary seeds, 0–29.

The raw behavior explains the table. Across all 393 full-run steps, the model gave one unique raw response:

```
Down
```

That is semantically close. It is also format-invalid. The parser required `<answer>Down</answer>`, so every step fell back to Right.

I also checked the obvious relaxed-parser counterfactual. Since every raw response was exactly `Down`, no LLM parser is needed: a human parser would turn every response into the action Down. That gives an always-Down policy, not a random policy. It still solved 0/30 maps on both seed sets. On its own trajectory, the action was usually either a wall bump or a poor local choice, as Table 3 shows. Here "GT greedy" means matching the one-step GT greedy planner at that visited state.

Table 3. Relaxed human-parser diagnostic for the always-Down policy.

Metric	Preliminary	Held-out
Success	0/30	0/30
GT greedy match	3.0% (12/403)	2.5% (34/1366)
Safe/goal	24.1% (97/403)	10.2% (140/1366)
Wall	69.2% (279/403)	88.5% (1209/1366)

Table 4. Dataset summary. The split unit is map seed, not transition row.

Item	Value
Reachable maps	100
Map seeds	0–99
Train maps	80, seeds 0–79
Validation maps	20, seeds 80–99
Transitions per policy/map	15
Train rows	2400
Validation rows	600
Total rows	3000

On the held-out comparison seeds 100–129, perfect one-step GT transitions produced 6/30 successes, 0/30 holes, and 24/30 truncations. The mean episode length was 82.80 steps.

This is not a world-model failure. The predictor is perfect. It is a planner failure: one-step Manhattan lookahead avoids immediate holes, but it can loop or miss detours.

3.4. Verdict

Part 1 gives two warnings. First, zero-shot VLM control can fail at the interface before it fails at reasoning. Second, the GT one-step-greedy planner is only a ceiling for this specific myopic planner. That ceiling is low.

4. Building the Transition Dataset

The world model learns:

```
(image_t, action_t)
  -> (next_position, outcome)
```

This is the part where label mistakes would poison everything downstream. I checked the deterministic transition labels against actual Gymnasium stepping.

The map-level split matters. All transitions from one map

Table 5. Outcome distribution. Terminal outcomes are sparse, especially in validation.

Split	Safe	Hole	Goal	Wall
Train	1886	130	22	362
Validation	473	29	5	93

stay in one split, so validation does not reuse train maps.

I collected from two policies. Greedy rows look like plausible route-following behavior. Random rows add more walls, holes, and off-route states. That matters because a transition model trained only on clean greedy paths would not see enough unsafe outcomes.

One target looked like this:

```
<prediction>
Position: (0, 1). Outcome: safe
</prediction>
```

For the SFT image + GT state text condition, the model also receives symbolic state text:

```
State: player_position=(0, 0);
goal_position=(7, 7);
hole_positions=[...]
```

Verification checks passed: split integrity, target parsing, and Gymnasium transition label checks.

5. Part 2: Training a VLM World Model

5.1. Hypothesis

GT state text should help if reading the board from pixels is the hard part. With text, the model only needs to learn the transition rule. Without text, it has to infer the state from the image and apply the rule.

5.2. Training Setup

Both SFT runs used `Qwen/Qwen2.5-VL-3B-Instruct` with PEFT LoRA on Colab A100.

Final losses are not the whole learning curve. Figure 1 shows the SFT image + GT state text run dropping quickly; SFT image-only starts higher and decays more gradually.

5.3. Calibration Gate

Before trusting validation accuracy, I checked train accuracy. The rule was simple: if train both-correct accuracy (position and outcome both correct) is far below 90%, do not interpret validation as meaningful.

Both models passed. SFT image + GT state text reached 1.0000 train both-correct; SFT image-only reached 0.9992.

Table 6. SFT setup. Both conditions used the same training recipe.

Setting	Value
Method	PEFT LoRA
LoRA rank / alpha / dropout	16 / 32 / 0.05
Trainable parameters	37,152,768
Trainable percent	0.9798%
dtype	bf16
GPU	NVIDIA A100-SXM4-80GB
Epochs	2
Batch size	8
Learning rate	2e-4
Train / val rows	2400 / 600

Table 7. Final SFT losses. The learning curve is shown in Figure 1.

Condition	Train loss	Eval loss	Runtime
SFT image + GT text	0.002746	0.000003708	3295.1s
SFT image-only	0.025151	0.0010736168	3126.9s

So the validation metrics are interpretable. This does not prove out-of-distribution generalization. It says the models fit the assignment distribution.

5.4. One-Step Evaluation

SFT image + GT state text was perfect on the measured train and validation splits. SFT image-only was still strong, but not perfect.

The SFT image-only errors are small: safe vs hole, hole vs safe, and one wall case predicted as safe. I would not draw a strong perception conclusion from validation alone. The safer reading is narrower: image-only is slightly less perfect on this split, while SFT image + GT state text gives the model symbolic player, goal, and hole positions directly.

5.5. Verdict

Part 2 worked on the assignment's one-step generated-map distribution. GT state text helped, but the gap was small: 100.0% both-correct for SFT image + GT state text versus 99.17% for SFT image-only on validation.

The numbers are strong on the assignment distribution, but terminal outcomes are sparse: only 5 validation goal examples and 29 hole examples. I would not call this a broad solution to dynamics. The claim is narrower: it covers this fixed one-step task, map generator, and output format.

The more important lesson appears in Part 3. A one-step model can be very accurate offline and still be fragile inside a closed-loop planner.

Table 8. Main one-step validation metrics. Both-correct means position and outcome are both correct.

Condition	Split	Count	Format	Exact	Pos.	Outcome	Both	Wrong pos.	Wrong out.	Format err.
SFT image + GT state text	train	2400	1.0000	1.0000	1.0000	1.0000	1.0000	0	0	0
SFT image + GT state text	val	600	1.0000	1.0000	1.0000	1.0000	1.0000	0	0	0
SFT image-only	train	2400	1.0000	0.9992	1.0000	0.9992	0.9992	0	2	0
SFT image-only	val	600	1.0000	0.9917	0.9950	0.9917	0.9917	3	5	0

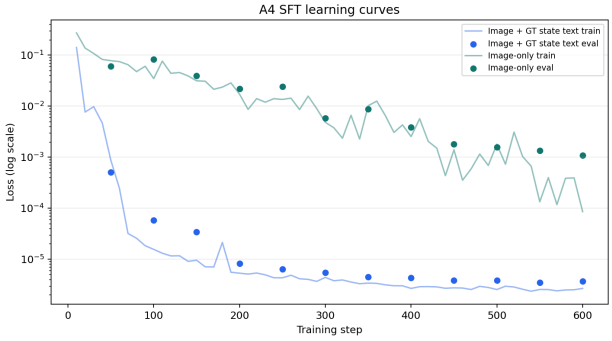

Figure 1. SFT learning curves.

Table 9. Validation outcome confusion, SFT image + GT state text. Rows are ground-truth outcomes; columns are predicted outcomes.

Target	Safe	Hole	Goal	Wall
safe	473	0	0	0
hole	0	29	0	0
goal	0	0	5	0
wall	0	0	0	93

6. Part 3: Planning in Imagination

6.1. Hypothesis

Before running the held-out planning comparison, I expected a clean ordering: GT one-step lookahead should be the ceiling, SFT image + GT state text should come next, SFT image-only should be lower because it has to read the board from pixels, and both zero-shot conditions should mostly fail at the strict output contract. I also expected the GT one-step-greedy ceiling to be fairly high, roughly 70–90%, because the planner could reject immediate holes and walls. That last expectation was wrong.

6.2. Design

The planner is deliberately simple. For each real state: render the current image, ask the predictor for the next position and outcome for each action, reject predicted holes and walls, choose the remaining action closest to the GT goal by Manhattan distance, and execute that action in the real environment.

The predictor can be zero-shot Qwen, one of the SFT models, or GT transition logic. The VLM does not choose the action directly in lookahead conditions. It predicts one-step dynamics only.

Table 10. Validation outcome confusion, SFT image-only. Rows are ground-truth outcomes; columns are predicted outcomes.

Target	Safe	Hole	Goal	Wall
safe	471	2	0	0
hole	2	27	0	0
goal	0	0	5	0
wall	1	0	0	92

For `lookahead_sft_image_text`, each one-step query used the current rendered image plus GT state text recomputed from the real environment state. For `lookahead_sft_image_only`, each query used the current rendered image and action only, with no GT state text.

All five planning-comparison conditions use the same held-out seeds, 100–129. These maps are separate from the training seeds 0–79 and validation seeds 80–99. For a fair comparison, the reactive baseline is rerun on the same held-out seed set.

6.3. Results

Metric denominators: success, hole, and truncation rates are per episode over 30 seeds. Mean steps is environment steps per episode. Format compliance and parse failure are per model output/query. Reactive uses one model query per environment step; lookahead uses four action queries per environment step. Fallback rate is per environment decision step.

For binary success and hole rates, I use Wilson 95% intervals. The original bootstrap intervals were degenerate for some 0-count rates, which is misleading at $n = 30$. Mean-step intervals remain bootstrap intervals.

The ordering roughly matches the hypothesis, but the ceiling is much lower than I expected. SFT image + GT state text is the best observed learned condition at 16.7% success, or 5/30 maps. SFT image-only reaches 10.0%, or 3/30. The GT one-step-greedy planner reaches 20.0%, or 6/30. With 30 seeds, the confidence intervals are wide, so I would not claim a clean statistical separation between the learned conditions. The failure modes matter more than the small ranking.

Table 11. Main planning metrics on held-out seeds 100–129.

Condition	Episodes	Successes	Success	Wilson 95% CI	Mean steps	Hole	Trunc.
reactive_vlm	30	0/30	0.0000	[0.000, 0.114]	22.50	0.8000	0.2000
lookahead_zeroshot_vlm	30	0/30	0.0000	[0.000, 0.114]	22.50	0.8000	0.2000
lookahead_sft_image_text	30	5/30	0.1667	[0.073, 0.336]	59.03	0.3000	0.5333
lookahead_sft_image_only	30	3/30	0.1000	[0.035, 0.256]	17.40	0.8000	0.1000
lookahead_gt	30	6/30	0.2000	[0.095, 0.373]	82.80	0.0000	0.8000

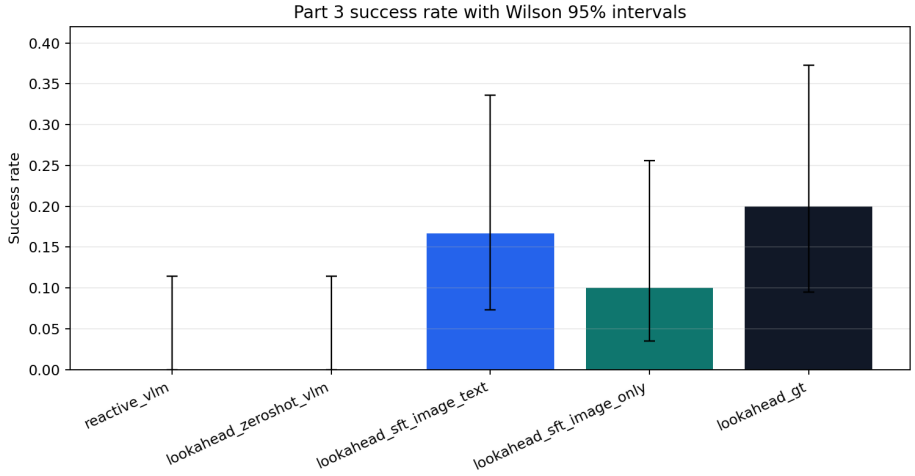

Figure 2. Part 3 success rates with Wilson confidence intervals. At $n = 30$, the intervals are wide.

Table 12. Interface diagnostics. Format and parse metrics are per model output/query; fallback is per environment decision.

Condition	Format	Parse fail	Fallback	Runtime
reactive_vlm	0.0000	1.0000	1.0000	156.3
zeroshot_lookahead	0.0000	1.0000	1.0000	575.0
sft_image_text	1.0000	0.0000	0.0000	3813.3
sft_image_only	1.0000	0.0000	0.0000	1120.2
lookahead_gt	1.0000	0.0000	0.0000	6.8

Table 13. Closed-loop prediction diagnostics from saved planning traces.

Condition	All correct	All total	Sel. correct	Sel. total
sft_image_text	6368	7084	1124	1771
sft_image_only	1981	2088	497	522

6.4. What Went Wrong

Zero-shot failed at the interface again. Reactive Qwen returned strings like `Down`. Zero-shot world-model Qwen returned strings like:

```
Position: (0, 1). Outcome: safe
```

Both are close to useful. Both are invalid under the strict parser. The measured strict-format compliance is 0.0000 for both zero-shot conditions. I keep that as the assignment metric, then separate it from semantic intent with a relaxed manual-parser diagnostic for the reactive outputs.

A human or LLM-based parser can read `Down`. The strict parser cannot accept it under the declared contract. I therefore added the relaxed always-Down diagnostic in Table 3: even if `Down` is accepted as an action, it still solves 0/30 maps on both seed sets.

The SFT models fixed the format contract. Both SFT planning conditions had format compliance 1.0000 and parse failure 0.0000. After that, the important closed-loop questions are prediction quality and planning.

Closed-loop prediction diagnostics are less clean than iid validation metrics, but they are useful. I recomputed the true deterministic transition for every saved lookahead query and compared it with the model prediction in the trace.

The selected-action accuracies are 0.6347 for SFT image + GT state text and 0.9521 for SFT image-only. This is a diagnostic, not the headline. These are closed-loop visited-state distributions, not iid validation rows. The inversion does not mean SFT image-only is the better world model: SFT image + GT state text ran about 3.4× longer by mean episode length, so it accumulated many repeated-loop queries outside the training distribution. Direct comparison is messy. Possible explanations include on-policy distribution shift, repeated-state artifacts, prompt/context mismatch, the planner selecting actions where mistakes matter most, and limits

of this trace-labeling diagnostic. The useful point is smaller: offline one-step validation accuracy did not transfer cleanly to every state visited by the planner.

The safety-critical errors are more revealing than aggregate both-correct accuracy. A false-safe prediction means the model says an action is usable, while the true transition is a `hole` or `wall`. A filter-based planner cannot recover from those if it selects the action.

For SFT image + GT state text, every hole episode ended with a selected false-safe action: 8 final steps predicted `goal` but were truly `hole`, and 1 predicted `safe` but was truly `hole`. For SFT image-only, all 24 hole episodes ended with final predictions of `safe` for true holes. Truncations are different: SFT image + GT state text had 9 truncation episodes with no selected model error and 7 with selected model errors; SFT image-only had 3 truncations, all with no selected model error.

The GT condition exposes the planning problem. It never steps into a hole, so its 0.0000 hole rate is reassuring. But it truncates on 80.0% of maps. Seed 100 is a representative failure: perfect one-step predictions, 100 steps, no goal. The planner can avoid immediate danger but still fail to make progress.

This was not because the held-out maps were impossible. As a CPU-only diagnostic, I ran BFS over non-hole cells for seeds 100–129. All 30 maps were reachable, all had shortest paths within the 100-step cap, the mean shortest path was 14.13 steps, and the maximum was 16. The low GT result is about the GT one-step-greedy planner, not map impossibility.

This is the key negative result. Moving planning out of the VLM does not make planning good by itself. The external planner still has to be a good planner.

7. What Actually Bottlenecked?

Two main issues show up in the experiments.

First: output formatting. Zero-shot Qwen gave near-miss strings rather than the required tags. I report that as a formatting violation under the strict contract, not as evidence that Qwen cannot choose an action. The relaxed manual-parser check keeps those two questions separate: accepting `Down` as an action still gives an always-Down policy with 0/30 successes.

Second: one-step greedy planning. GT dynamics shows that one-step greedy planning has a low ceiling; learned-model planning is limited by both on-policy prediction errors and planner myopia. Perfect one-step GT predictions only reached 20.0% success. The planner is greedy, has no memory, and does not search over routes. It optimizes the next predicted Manhattan distance, not the path.

The model learned the local physics. The planner did not learn a global strategy. I mean that in the narrow sense supported here: offline validation showed strong one-step prediction on the assignment distribution, while the GT one-step-greedy ceiling and closed-loop traces showed that local predictions were not enough for reliable route-finding.

8. Limitations and Future Work

This is a homework report, not a broad benchmark. The limitations are concrete. Part 3 uses 30 maps, so the confidence intervals are wide. Train and validation maps come from the same generator distribution. The relaxed always-Down check is only a diagnostic for semantic intent; it is not the strict assignment metric. The planner is only one-step greedy. It has no visited-state penalty, no graph search, no value function, and no uncertainty handling.

I also did not implement the bonus experiments: no H=2/H=3 lookahead, no CoT planning, no 4×4 comparison. No bonus experiments (A, B, or C) were attempted.

The obvious next step is not "train harder." The one-step validation metrics are already high. Better diagnostics would be richer relaxed-parser zero-shot evaluation, H=2 or H=3 lookahead, a visited-state penalty or BFS-style planner using the learned model, uncertainty calibration for unsafe predictions, and a 4×4 vs 8×8 comparison to separate map scale from model quality.

9. Conclusion

The answer is split.

Yes, a VLM can learn to imagine one-step FrozenLake transitions on this same generated-map distribution and fixed post-SFT output format. In this setup, supervised Qwen2.5-VL adapters reached 100.0% validation both-correct accuracy for SFT image + GT state text and 99.17% for SFT image-only.

No, that did not make the planner strong. The best observed learned planning condition reached 16.7% success, and the perfect GT one-step-greedy planner reached only 20.0%. That ceiling changes how I read the rest of the results.

The blunt version is: on this one-step task, the model learned the physics; the planner still got lost.

Reproducibility Notes

Main scripts are `run_a2_reactive_vlm.py`, `collect_data.py`, `train_sft.py`, `eval_world_model.py`, `run_planning.py`, and `plot_results.py`. Main source artifacts include the final metric tables, confusion matri-

Table 14. Safety-critical closed-loop errors. False-safe true-hole errors are the fatal ones.

Condition	Sel. false-safe hole	Sel. false-safe wall	All false-safe hole	All false-safe wall	False-reject safe	Hole final false-safe
lookahead_sft_image_text	9	638	16	700	0	9/9
lookahead_sft_image_only	24	1	46	45	8	24/24

Table 15. Seed 100 GT one-step-greedy loop.

Step	Position	Action	Next position
9	(2, 7)	Left	(2, 6)
10	(2, 6)	Right	(2, 7)
11	(2, 7)	Left	(2, 6)

ces, planning traces, summaries, and derived plots under `results/`.